

Грамматика продвинутого лямбда-интерпретатора

Требования к языку

Входная программа состоит из набора именованных определений и одного лямбда-выражения. Каждое определение записывается на отдельной непустой строке и имеет вид

```
let name = expression
```

и могут использоваться в последующем выражении как обычные переменные. Слово `let` является ключевым словом и не может быть именем переменной или определения.

Лямбда обозначается символом `\`. Должны поддерживаться абстракции с несколькими параметрами:

```
\x y z. expression
```

Такая запись является сокращением для вложенных абстракций:

$$\lambda x. \lambda y. \lambda z. \text{expression}.$$

Лексические правила

Пробельные символы между токенами игнорируются. Идентификатор начинается с буквы, далее могут идти буквы, цифры, апостроф и символ подчеркивания. Ключевое слово `let` исключается из множества идентификаторов.

```
letter      ::= "a" | ... | "z" | "A" | ... | "Z"
digit       ::= "0" | ... | "9"
identifier  ::= letter (letter | digit | "'" | "_")*
```

Факторизованная грамматика

Грамматика записана в EBNF. Оператор `+` означает «один или больше», оператор `*` используется только в лексическом правиле идентификатора. В синтаксической части нет явных ϵ -продукций.

```
program      ::= definition* expression
definition   ::= "let" identifier "=" expression lineEnd
expression   ::= abstraction | applicationChain
abstraction  ::= "\" identifier+ "." expression
applicationChain ::= atom+
atom         ::= identifier | "(" expression ")"
lineEnd      ::= end of current non-empty line
```

Почему грамматика подходит для парсера

В исходной грамматике аппликация естественно записывается как

$$\text{term} ::= \text{term term},$$

что дает левую рекурсию. Для `FParser` и рекурсивного спуска такая форма неподходящая: парсер `term` немедленно вызывает сам себя без потребления входа.

В предложенной грамматике аппликация заменена на цепочку атомов:

$$applicationChain ::= atom^+.$$

Такой нетерминал парсится как `many1 atom`. Если в цепочке один атом, результатом является сам атом. Если атомов несколько, AST строится левоассоциативно:

$$a\ b\ c \equiv ((a\ b)\ c).$$

Альтернативы в правиле `expression` различаются первым токеном: абстракция начинается с `\`, а цепочка аппликации начинается с идентификатора или открывающей скобки. Поэтому правило удобно реализовать через `choice` без попыток угадывать одинаковые префиксы.

Разделение определений по строкам нужно для однозначности. Без этого запись

```
let K = \x y.x
S K K
```

можно было бы прочесть как одно определение с телом `\x y.x S K K`. Поэтому на уровне программы непустые строки, начинающиеся с ключевого слова `let`, считаются определениями, а оставшиеся строки образуют итоговое выражение.

Построение AST

Для дальнейшей реализации удобно использовать тот же базовый AST, что и в предыдущей задаче:

$$\begin{aligned} Expr &::= Var(name) \\ &| Abs(name, Expr) \\ &| App(Expr, Expr) \end{aligned}$$

Именованные определения могут храниться отдельно в таблице

$$name \mapsto Expr.$$

Перед нормализацией входного выражения имена из таблицы можно раскрыть или учитывать их при редукции как заранее заданные переменные.

Абстракция с несколькими параметрами строится как последовательность вложенных `Abs`. Например:

```
\x y.x -> Abs(x, Abs(y, Var(x)))
```

Цепочка аппликации строится левым сворачиванием. Например:

```
S K K -> App(App(Var(S), Var(K)), Var(K))
```

Пример 1

Вход:

```
let S = \x y z.x z (y z)
let K = \x y.x
```

```
S K K
```

Разбор определений:

$$\begin{aligned} S &\mapsto Abs(x, Abs(y, Abs(z, App(App(Var(x), Var(z)), App(Var(y), Var(z))))) \\ K &\mapsto Abs(x, Abs(y, Var(x))) \end{aligned}$$

Разбор итогового выражения:

$$S \ K \ K \rightarrow App(App(Var(S), Var(K)), Var(K))$$

После раскрытия определений и бета-редукции ожидаемый результат:

$\lambda x. x$

или любой альфа-эквивалентный терм.

Пример 2

Вход:

$let\ I = \lambda x. x$

$(\lambda x. x)\ I$

Разбор итогового выражения:

$$(\lambda x. x)\ I \rightarrow App(Abs(x, Var(x)), Var(I))$$

После раскрытия определения I и бета-редукции:

$\lambda x. x$